

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ
по лабораторной работе №4
по курсу «Логика и основы алгоритмизации
в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили ст. гр. 25BBB2:

Карпов А.А.

Долгов Л.А.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., доцент Деев М.В.

Пенза 2026

Тема: Бинарное дерево поиска

Цель работы: изучить организацию бинарного дерева поиска как динамической структуры данных и получить практические навыки реализации на его основе операций поиска элемента и подсчёта числа его вхождений.

Лабораторные задания: 1) реализовать алгоритм поиска вводимого с клавиатуры значения в уже построенном дереве; 2) реализовать функцию подсчёта числа вхождений заданного элемента в дерево; 3*) изменить функцию добавления элементов для исключения добавления одинаковых значений; 4*) оценить сложность процедуры поиска по значению в бинарном дереве.

Средства реализации и проверки: программы реализованы на языке Си с использованием динамического выделения памяти (malloc) и рекурсивных функций; корректность работы проверялась многократным запуском с разными входными данными, а оценка сложности поиска — отдельной программой, строящей деревья разного размера и подсчитывающей их высоту и среднее число сравнений при поиске.

Теоретические сведения

Бинарное дерево — это динамическая структура данных, каждый узел которой имеет не более двух потомков, называемых левым и правым. Узел содержит информационное поле и два указателя — на левое и на правое поддереву; отсутствующий потомок обозначается значением NULL. Бинарное дерево поиска дополнительно упорядочено: для каждого узла все значения в одном из его поддеревьев меньше значения самого узла, а все значения в другом поддереве — больше. Такое упорядочение позволяет хранить данные в отсортированном виде и выполнять поиск значительно быстрее, чем в неупорядоченном списке.

В использованной реализации (заданной в методических указаниях и сохранённой без изменений в части построения дерева) принято соглашение, зеркальное по отношению к привычному из учебников: в левое поддерево помещаются значения, большие значения родителя, а в правое — меньшие либо равные. Само по себе это не нарушает свойство упорядоченности — дерево

остаётся корректным бинарным деревом поиска, просто «отражённым» слева направо. Более того, такой выбор согласован с функцией вывода дерева на экран: она печатает сначала правое поддереву, затем сам узел, затем левое, и из-за зеркального соглашения это даёт на экране значения по возрастанию сверху вниз, что и подтверждается рисунками 1 и 2 далее.

Так как дерево — рекурсивная структура (каждое поддереву само является деревом), операции над ним — построение, поиск, подсчёт вхождений, обход — естественно реализуются рекурсивными функциями: функция обрабатывает текущий узел и вызывает саму себя для левого и правого поддеревьев.

Задания 1 и 2. Поиск значения и подсчёт числа вхождений

Организация алгоритмов

Функция поиска значения рекурсивно спускается по дереву: если текущий узел пуст, значение не найдено; если значение в текущем узле совпадает с искомым, узел найден; иначе в соответствии с принятым в дереве соглашением (большие значения — налево, меньшие — направо) поиск продолжается в соответствующем поддереве. На каждом шаге рассматривается только одно из двух поддеревьев, поэтому путь поиска — это путь от корня до одного конкретного узла (либо до пустого указателя, если значение отсутствует).

Функция подсчёта числа вхождений устроена похожим образом, но не останавливается при первом совпадении: поскольку при построении дерева одинаковые значения всегда уходят в одно и то же поддерево (равные — туда же, куда и меньшие), при обнаружении совпадения функция прибавляет единицу к счётчику и продолжает поиск в этом поддереве, где могут находиться остальные экземпляры того же значения.

Проверка работы программы

Было построено дерево из значений 10, 5, 15, 3, 7, 12, 18 с намеренно добавленным повторно значением 7, после чего выполнен поиск и подсчёт вхождений для числа 7. Результат показан на рисунке 1.

```
tree1.exe — Поиск и подсчёт вхождений

-1 - окончание построения дерева
Введите число: 10
Введите число: 5
Введите число: 15
Введите число: 3
Введите число: 7
Введите число: 12
Введите число: 18
Введите число: 7
Введите число: -1
Построение дерева окончено

Ваше дерево:
  3
  5
  7
  7
10
 12
 15
 18

--- ЗАДАНИЕ 1 И 2 ---
Введите число для поиска и подсчета вхождений: 7
Элемент 7 успешно НАЙДЕН в дереве!
Число вхождений элемента 7 равно 2
```

Рисунок 1 — Поиск значения и подсчёт числа его вхождений в дереве

Как видно из рисунка 1, значения в построенном дереве действительно выводятся по возрастанию сверху вниз (3, 5, 7, 7, 10, 12, 15, 18), что подтверждает описанное выше зеркальное соглашение о размещении потомков. Элемент 7 найден, а число его вхождений правильно определено равным двум — ровно столько раз это значение вводилось при построении дерева. Дополнительно была проверена и обратная ситуация — поиск отсутствующего в дереве значения корректно возвращает отрицательный результат и нулевое число вхождений.

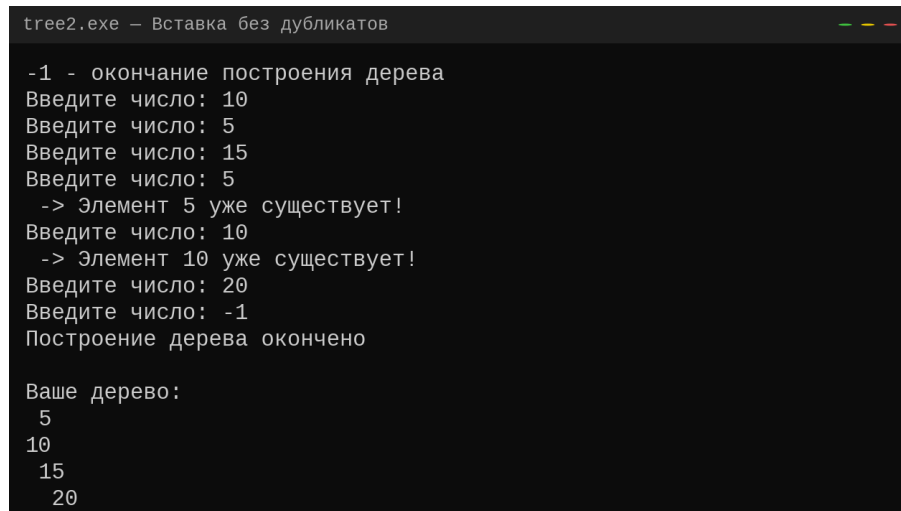
Задание 3*. Исключение повторяющихся значений

Организация алгоритма

Функция добавления элемента изменена так, чтобы различать три случая при спуске по дереву: новое значение больше текущего узла — переход в поддереву с большими значениями; строго меньше — переход в поддереву с меньшими значениями; равно значению текущего узла — новый элемент не добавляется, а на экран выводится сообщение о том, что такое значение в дереве уже есть. Благодаря этому в дереве гарантированно не может появиться двух узлов с одинаковым значением.

Проверка работы программы

В дерево были добавлены значения 10, 5, 15, после чего предпринята попытка повторно добавить уже имеющиеся значения 5 и 10, и затем добавлено новое значение 20. Результат показан на рисунке 2.



```
tree2.exe - Вставка без дубликатов
-1 - окончание построения дерева
Введите число: 10
Введите число: 5
Введите число: 15
Введите число: 5
-> Элемент 5 уже существует!
Введите число: 10
-> Элемент 10 уже существует!
Введите число: 20
Введите число: -1
Построение дерева окончено

Ваше дерево:
5
10
15
20
```

Рисунок 2 — Вставка элементов с исключением дубликатов

Как видно из рисунка 2, обе попытки повторной вставки (значений 5 и 10) были корректно отклонены с выводом соответствующего сообщения, при этом ни одно из них не было добавлено в дерево повторно — в итоговом дереве присутствуют только уникальные значения 5, 10, 15 и 20.

Задание 4*. Оценка сложности поиска

Теоретическая оценка

На каждом шаге поиска выполняется ровно одно сравнение и переход ровно в одно из двух поддеревьев, поэтому число сравнений при поиске совпадает с длиной пути от корня до искомого узла и не превышает высоты дерева h . Таким образом, сложность поиска по значению в бинарном дереве поиска составляет $O(h)$.

Высота дерева, в свою очередь, зависит от порядка добавления элементов и в общем случае деревом не контролируется (балансировка в рассматриваемой реализации не производится). Если элементы добавляются в случайном порядке, дерево получается в среднем сбалансированным и его высота растёт как $O(\log_2 N)$, где N — число элементов; соответственно и сложность поиска в

среднем составляет $O(\log_2 N)$. Если же элементы добавляются в уже отсортированном порядке, каждый новый элемент становится потомком предыдущего с одной и той же стороны, и дерево вырождается в линейный список: его высота становится равна N , а сложность поиска в худшем случае — $O(N)$, то есть не лучше, чем линейный перебор массива.

Экспериментальное подтверждение

Для подтверждения теоретической оценки была написана отдельная программа, строящая дерево из N уникальных значений двумя способами — в случайном порядке и в отсортированном порядке — и измеряющая высоту получившегося дерева, а также среднее число сравнений при поиске 500 случайных значений, уже находящихся в дереве. Результаты для нескольких размеров N сведены в таблице 1 и показаны на рисунке 3.

Таблица 1 — Высота дерева и среднее число сравнений при поиске

N	Высота (случ.)	Высота (отсорт.)	Сравнений (случ.)	Сравнений (отсорт.)
100	14	100	8,2	51,9
1000	24	1000	12,1	520,5
5000	28	5000	15,9	2494,0
10000	31	10000	17,5	5146,6
20000	33	20000	18,3	9958,1

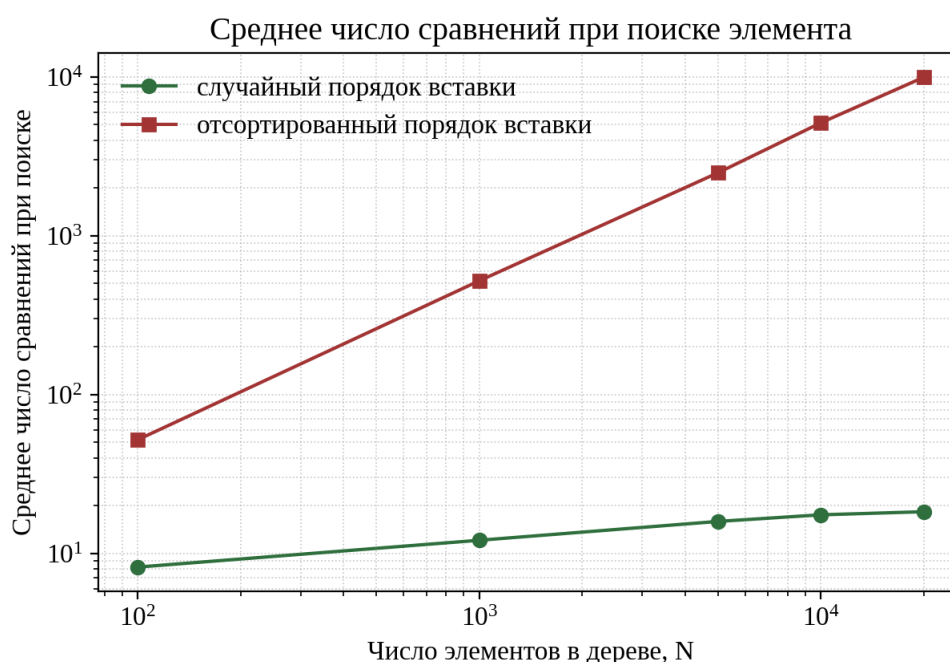


Рисунок 3 — Среднее число сравнений при поиске в зависимости от N (логарифмический масштаб по обеим осям)

Результаты полностью подтверждают теоретическую оценку. При отсортированном порядке вставки высота дерева во всех случаях в точности равна N — дерево действительно вырождается в линейный список, а среднее число сравнений при поиске растёт пропорционально N (на рисунке 3 эта зависимость выглядит прямой линией в логарифмическом масштабе по обеим осям, что характерно именно для линейного роста). При случайном порядке вставки высота дерева растёт значительно медленнее — с 14 при $N = 100$ до 33 при $N = 20000$, то есть почти в 200 раз меньший прирост высоты при 200-кратном росте N , — а среднее число сравнений при поиске остаётся в пределах 8–19 на всём исследованном диапазоне, что соответствует логарифмической зависимости от N .

Таким образом, одна и та же реализация бинарного дерева поиска может вести себя принципиально по-разному в зависимости от порядка поступления данных: на случайных данных поиск выполняется за $O(\log_2 N)$, а на уже отсортированных — вырождается до $O(N)$. Это объясняется отсутствием в рассматриваемой реализации самобалансировки (такой, как в АВЛ-деревьях или красно-чёрных деревьях), которая не позволила бы дереву выродиться в линейный список независимо от порядка вставки элементов.

Вывод

В ходе выполнения работы изучена организация бинарного дерева поиска и получены практические навыки реализации на его основе операций поиска значения и подсчёта числа его вхождений. Экспериментальная проверка подтвердила корректность реализованных алгоритмов: поиск правильно определяет наличие или отсутствие значения в дереве, а подсчёт вхождений верно учитывает все повторы искомого значения. Дополнительно реализован и проверен вариант функции добавления, исключающий появление в дереве повторяющихся значений.

Теоретическая оценка сложности поиска — $O(h)$, где h — высота дерева, — подтверждена экспериментально: при случайном порядке добавления

элементов высота дерева и число сравнений при поиске растут логарифмически от числа элементов, тогда как при добавлении уже отсортированных данных дерево вырождается в линейный список высотой N , и сложность поиска ухудшается до $O(N)$. Из этого следует практический вывод: эффективность простого (небалансирующегося) бинарного дерева поиска существенно зависит от порядка поступления входных данных.